Phase 1: Environment and Project Initialization

1. **Project Structure and Environment Setup:** Established the foundational project structure, necessary directories, and all dependencies. The selection of Docker was made to ensure a production-ready environment and provide inherent portability.
2. **Container Configuration:** Defined the `Dockerfile` and `docker-compose.yml` to manage services including PostgreSQL, Redis, pgAdmin, and the main application.
3. **Local Environment Provisioning:** Utilized the Docker command-line interface on macOS, in conjunction with Colima, to create a Linux-based environment configured with 6GB of memory, 4 CPU cores, and a 60GB disk allocation.
4. **Security and Integrity Measures:** Configured environment variables, `.gitignore`, and other critical files to maintain project integrity and security standards.

Phase 2: AdonisJS Project Bootstrap

- **Objective:** Initialize the AdonisJS v6 project within the Docker environment, install all requisite packages, configure PostgreSQL, Redis, and JSON Web Token (JWT) authentication, and validate the entire stack startup using a single `docker compose up` command.
- **Initialization Steps:**
    1. The AdonisJS installer was executed using a temporary Node container, obviating the need for a local Node.js installation.
    docker run --rm -it \
    2. -v $(pwd):/app \
    3. -w /app \
    4. node:20-alpine \
    5. sh -c "npm init adonisjs@latest . -- --kit=api --db=postgres --auth-guard=access_tokens --no-git-init"
        - `--kit=api`: Specifies an API-only project structure (without frontend views).
        - `--db=postgres`: Pre-configures the project for PostgreSQL database usage.
        - `--auth-guard=access_tokens`: Implements JWT-style token authentication.
        - `--no-git-init`: Skips Git initialization as it was previously configured.
    6. Completed the subsequent bootstrapping and setup procedures.

Phase 3: Robust Authentication System Implementation

- **Objective:** Develop a complete authentication system adhering to SOLID principles.
- **Core Functionalities:**
    - User Registration (utilizing bcrypt for password hashing).
    - Login (issuance of JWT access token).

    - Refresh Token Mechanism (stored in Redis for persistence).
    - Guest Token Generation (for anonymous users).
    - Logout Functionality (token invalidation).
    - Auth Middleware (for route protection and access control).
- **Authentication Module Architecture (SOLID Principles):** A layered architecture was implemented to separate concerns:
    - `AuthController` (HTTP Layer): Handles incoming requests and delegates to the service layer.
    - `IAuthService` (Interface/Contract): Defines the contract for the business logic.
    - `AuthService` (Business Logic): Contains the core authentication logic.
    - `IUserRepository` (Interface): Defines the contract for user data access.
    - `UserRepository` (Data Layer - Lucid ORM): Implements data access operations using PostgreSQL.
    - `Redis Service`: Utilized for refresh token storage and blacklisting.
- **Encountered Errors and Resolutions:**
    - **Error:** Corrupt file error during schema migration due to duplicate files.
        - **Fix:** Removed all corrupt database data and performed a complete rebuild of the environment.
    - **Error:** Login failure with valid credentials post-registration (a password hashing verification issue).
        - **Fix:** Replaced the dynamic `await import('@adonisjs/core/services/hash')` within the verification method with a direct use of the pre-imported `hash` service.

-----**Project Summary — Phases 0, 1 & 2**

🏗️ **Project:** ConvoCore — WhatsApp-Inspired Chat API

Phase 0 — Environment Setup Summary

- **Activities:** Verified installations (VS Code, Colima, Docker CLI), initialized Colima (4 CPU / 6GB RAM / 60GB disk), created GitHub repository and Git strategy (main → develop → feature/*), established project folder structure, created multi-stage `Dockerfile` and `docker-compose.yml` (4 services: `chat_app`, `chat_postgres`, `chat_redis`, `chat_pgadmin`), configured testing environment (`docker-compose.test.yml`), and set up environment files and ignore lists.
- **Errors & Fixes:** None reported.

Phase 1 — AdonisJS Project Bootstrap Summary

- **Activities:** Executed AdonisJS v6 installer via Docker, installed required packages (`pg`, `@adonisjs/redis`, `@adonisjs/transmit`, `bcryptjs`, `jsonwebtoken`), configured database, Redis, real-time (Transmit/SSE), authentication, and CORS settings, updated core configuration files (`adonisrc.ts`, `start/env.ts`, `start/kernel.ts`, `start/routes.ts`), created custom validator (`start/validator.ts`) and IoC provider (`providers/app_provider.ts`), and successfully booted the full Docker stack.
- **Errors & Fixes:**
    - **Error 1:** Incorrect installation subdirectory. **Fix:** Files were manually moved to the project root, and the extra directory was removed.
    - **Error 2:** Unnecessary session/shield providers generated for a REST API. **Fix:** Removed related packages and middleware from configuration.
    - **Error 3:** Missing import aliases in `package.json`. **Fix:** Added necessary aliases for `repositories` and `services/interfaces`.

Phase 2 — Authentication Module Summary

- **Functionality Built:** Implemented API endpoints for registration, login, token refresh, guest session creation, logout, and current user retrieval (`/api/v1/auth/*`).
- **Architecture:** Successfully implemented the SOLID-compliant architecture detailed above, including the creation of models, repositories (interfaces and concrete), services (interface and concrete), controllers, middleware, and validators.
- **Security Measures:** Employed `scrypt` hashing, configured JWT access (15 min) and refresh (7 days) tokens, implemented refresh token rotation, token blacklisting via Redis TTL, prevented user enumeration, and validated token types.
- **Errors & Fixes:**
    - **Error 1:** Duplicate migration files causing conflicts. **Fix:** All tables and migration history were dropped, all migration files were deleted, and new migrations were created with a corrected schema (UUID primary key, nullable email/password, `is_guest` column).
    - **Error 2:** Attempting to set a read-only `ctx.auth.user` property in the AuthMiddleware (AdonisJS v6 change). **Fix:** Stored the authenticated user on a custom `(ctx as any).authUser` property for retrieval in controllers.
    - **Error 3:** Login failing due to an issue with the `withAuthFinder` mixin querying nullable email columns. **Fix:** The `withAuthFinder` mixin was removed, and user verification was implemented manually in `AuthService` (find by email, manual password verification).
    - **Error 4:** Missing import alias for a repository. **Fix:** Added the correct alias to `package.json`.
- **Testing Status:** All specified authentication endpoints were tested and confirmed to be fully functional and returning the expected status codes (201, 200, 401, 409, 422).
- **Docker Stack Status:** All four Docker services (`chat_app`, `chat_postgres`,

`chat_redis`, `chat_pgadmin`) are running and reported as healthy or running.

Technology Stack:

- **Runtime:** Node.js 24 (Alpine)
- **Framework:** AdonisJS v6
- **Language:** TypeScript
- **Database:** PostgreSQL 16
- **Cache/Store:** Redis 7
- **ORM:** Lucid (`@adonisjs/lucid v22`)
- **Authentication:** JWT (`jsonwebtoken`) with Redis for token management
- **Hashing:** `scrypt` (via `@adonisjs/core/hash`)
- **Validation:** VineJS (`@vinejs/vine`)
- **Real-time:** `@adonisjs/transmit` (SSE) - Configured for later use
- **Containerization:** Docker + Colima (macOS)

Phase 3: **CONVERSATIONS MODULE**

Method	Endpoint	Description
`POST`	`/api/v1/conversati…`	Create a new conversation
`GET`	`/api/v1/conversati…`	Retrieve a list of the user's conversations
`GET`	`/api/v1/conversati…`	Retrieve a single conversation by ID
`DELETE`	`/api/v1/conversati…`	Delete a conversation
`POST`	`/api/v1/conversati…`	Add a participant to a conversation
`DELETE`	`/api/v1/conversati…`	Remove a participant from a conversation

Architectural Dependency Chain:

1. **ConversationController**
   - *Depends on* ↓
2. **IConversationService** (Interface)
   - *Implemented by* ↓
3. **ConversationService** (Business Logic Layer)

    - *Depends on ↓*
4. **IConversationRepository** (Interface)
    - *Implemented by ↓*
5. **ConversationRepository** (Data Access Layer, utilizing Lucid ORM)

Key Deliverables:

The Conversations Module was implemented, encompassing the following components:

- **Models:**
    - `Conversation` model: Defined with a UUID primary key, encompassing attributes for conversation type (direct/group), name, `avatarUrl`, and `createdBy`.
    - `ConversationParticipant` model: Includes the participant's `role` (admin/member) and a `last_read_at` timestamp for future integration of unread message counts.
- **Database Migrations:**
    - Creation of the `conversations` table, utilizing a UUID as the primary key, an enumeration for conversation type, and appropriate indexes.
    - Creation of the `conversation_participants` table, enforcing a unique constraint on the composite key of `(conversation_id, user_id)`.
- **Architecture (SOLID Principles Adherence):**
    - `IConversationRepository`: An interface created to facilitate Dependency Inversion.
    - `ConversationRepository`: Implementation of the repository interface, handling all database interactions (find, create, delete, participant management).
    - `IConversationService`: An interface defining the service contract.
    - `ConversationService`: Implementation of the service interface, containing the complete business logic.
    - `ConversationController`: The controller acting as a thin HTTP layer.
    - `ConversationAccessMiddleware`: Middleware responsible for participant verification.
- **Validation:**
    - Conversation validators implemented using VineJS.
- **Application Configuration Updates:**
    - `AppProvider`: Updated to include Inversion of Control (IoC) bindings for the conversation repository and service.
    - `start/kernel.ts`: Updated to register the `conversationAccess` named middleware.
    - `start/routes.ts`: Updated to define all conversation-related routes.

API Endpoints Implemented:

Method	Route	Description
`POST`	`/api/v1/conversati…`	Creates a new conversation.
`GET`	`/api/v1/conversati…`	Retrieves a list of the user's conversations.
`GET`	`/api/v1/conversati…`	Retrieves a single conversation by ID.
`DELETE`	`/api/v1/conversati…`	Deletes a conversation.
`POST`	`/api/v1/conversati…`	Adds a participant to a conversation.
`DELETE`	`/api/v1/conversati…`	Removes a participant from a conversation.

Business Rules Enforced:

- **Direct Conversations:** Must include exactly one other participant.
- **Self-Conversation Prevention:** Attempting to create a conversation with oneself results in a HTTP 422 Unprocessable Entity error.
- **Duplicate Prevention:** Creating a duplicate direct conversation returns the existing conversation (HTTP 200).
- **Group Conversations:**
    - Require a name (minimum 2 characters).
    - Require at least one other participant.
    - The creator is automatically assigned the `admin` role.
    - Other invited participants are assigned the `member` role.
    - Only administrators are authorized to add new participants to a group.
    - Only administrators are authorized to remove other participants.
    - Users are permitted to remove themselves from a group.
- **Deletion Policy:** Only the conversation creator is authorized to delete the conversation.
- **Access Control:** Non-participants are blocked from accessing all conversation-specific routes and receive a HTTP 403 Forbidden response.
- **Middleware Execution:** The `ConversationAccessMiddleware` is executed prior to all routes involving a conversation ID (`/:id`).

Architectural Layer Dependencies (SOLID):

- `ConversationController` (HTTP Layer)
    - *depends on* $\downarrow$

- `IConversationService` (Interface)
    - *implemented by* $\downarrow$
- `ConversationService` (Business Logic)
    - *depends on* $\downarrow$
- `IConversationRepository` (Interface) + `IUserRepository` (Interface)
    - *implemented by* $\downarrow$ $\quad$ *implemented by* $\downarrow$
- `ConversationRepository` (Lucid ORM) $\quad$ `UserRepository` (Lucid ORM)

Files Created:

- `app/models/conversation.ts`
- `app/models/conversation_participant.ts`
- `app/repositories/interfaces/i_conversation_repository.ts`
- `app/repositories/conversation_repository.ts`
- `app/services/interfaces/i_conversation_service.ts`
- `app/services/conversation_service.ts`
- `app/controllers/conversation_controller.ts`
- `app/middleware/conversation_access_middleware.ts`
- `app/validators/conversation_validator.ts`
- `database/migrations/..._create_conversations_table.ts`
- `database/migrations/..._create_conversation_participants_table.ts`

Files Updated:

- `providers/app_provider.ts`: IoC bindings for `IConversationRepository` and `IConversationService` added.
- `start/kernel.ts`: `conversationAccess` named middleware registered.
- `start/routes.ts`: All six conversation routes configured.

Error Log and Resolutions:

- No errors or major fixes were required; Phase 3 was completed successfully on the initial attempt.

Test Coverage Summary (All Tests Passing):

Test Case	Route	Result
Create direct conversation	`POST /conversations`	201 Created
Create group conversation	`POST /conversations`	201 Created

Attempt self-conversation	`POST /conversations`	422 E_SELF_CONVER…
Create duplicate direct conversation	`POST /conversations`	200 (Returns existing)
List user conversations	`GET /conversations`	200 (List with participan…
Get single conversation	`GET /conversations…`	200 (Single conversatio…
Get conversation (non-member)	`GET /conversations…`	403 E_NOT_PARTICIP…
Add participant	`POST /conversation…`	200 (Participant added)
Remove participant	`DELETE /conversati…`	200 (Participant removed)
Delete conversation	`DELETE /conversati…`	200 (Conversation dele…

Phase 4: Messages Module Development

Key Accomplishments:

- **Model Creation:** Developed the `Message` model, incorporating a UUID primary key, `type` field (supporting text, image, file, and system messages), `content`, `parentId` for threading, an `isEdited` flag, and `deletedAt` for soft deletion.
- **Database Migration:** Implemented the messages migration, establishing indexes on `conversation_id`, `sender_id`, and `created_at`, as well as a composite index on (`conversation_id`, `created_at`).
- **Repository Layer:** Defined the `IMessageRepository` interface, adhering to the Dependency Inversion Principle. The concrete `MessageRepository` was implemented with capabilities for paginated queries, soft deletion, marking messages as read, and calculating unread counts.

- **Service Layer:** Established the `IMessageService` interface as the service contract. The `MessageService` was developed to encapsulate the full suite of business logic.
- **Controller Layer:** Created the `MessageController`, serving as a minimal HTTP interface.
- **Validation:** Utilized VineJS to create comprehensive validators for message operations (send, edit, list).
- **Inversion of Control (IoC):** Updated the `AppProvider` to include IoC bindings for the message repository and service.
- **Routing:** Modified `start/routes.ts` to include all five message-related routes, nested under `/conversations`.

API Endpoints Implemented:

Method	Path	Description
`POST`	`/api/v1/convers…`	Transmits a new message.
`GET`	`/api/v1/convers…`	Retrieves a list of messages.
`PATCH`	`/api/v1/convers…`	Modifies an existing message.
`DELETE`	`/api/v1/convers…`	Initiates a soft delete of a message.
`POST`	`/api/v1/convers…`	Marks a message as read.

Features Implemented:

- Support for multiple message types: Text, image, file, and system.
- Implementation of threaded replies via the `parentId` attribute.
- Soft deletion mechanism: Content is replaced with a placeholder, while the record is retained in the database.
- Edit tracking: The `isEdited` flag is set upon message modification.

- Pagination: Support for both cursor-based pagination (using the `before` message ID parameter) and page-based pagination (using `page` and `limit` parameters).
- Enforcement of a maximum limit of 100 messages per page within the service layer.
- Unread count tracking utilizing the `last_read_at` field on participant records.
- The 'Mark as Read' operation updates the participant's `last_read_at` timestamp.

Governing Business Rules:

- Message editing is restricted exclusively to the original sender (Unauthorized - 403 error).
- Message deletion is restricted exclusively to the original sender (Unauthorized - 403 error).
- Attempting to edit a message that has been deleted is prohibited (Unprocessable Entity - 422 error).
- System messages are not editable (Unprocessable Entity - 422 error).
- Deleted messages are displayed with a "This message was deleted" placeholder.
- A reply's parent message must belong to the same conversation.
- All message routes are secured by authentication and the `conversationAccess` middleware.

Files Created:

- `app/models/message.ts`
- `app/repositories/interfaces/i_message_repository.ts`
- `app/repositories/message_repository.ts`
- `app/services/interfaces/i_message_service.ts`
- `app/services/message_service.ts`
- `app/controllers/message_controller.ts`
- `app/validators/message_validator.ts`
- `database/migrations/..._create_messages_table.ts`

Files Updated:

- `providers/app_provider.ts`: Added bindings for `IMessageRepository` and `IMessageService`.
- `start/routes.ts`: Integrated all five message-related routes.

Technical Issues and Resolutions:

- **ISSUE 1: Incorrect `isDeleted` Status:** The `isDeleted` flag was erroneously appearing as `TRUE` for newly created messages.
    - *Root Cause:* The `message.deletedAt !== null` check was incorrectly evaluated because the Lucid ORM returns `deletedAt` as a raw value before its

cast to a DateTime object.
    - *Resolution:* The check was formally corrected to use `instanceof DateTime`: `const isDeleted = message.deletedAt instanceof DateTime`. The necessary `DateTime` import from `luxon` was also added to `message_service.ts`.
- **ISSUE 2: Terminal Test Failure for Threaded Replies:** A test for reply threading failed with the error: `"zsh: event not found: \\"`.
    - *Root Cause:* The `zsh` shell was incorrectly interpreting the `!` character within double-quoted strings as a history expansion command, which was not a code defect.
    - *Resolution:* A robust heredoc temporary file approach was adopted for passing the JSON payload:
    cat > /tmp/reply.json << EOF
    - { "content": "...", "parentId": "$MSG_ID" }
    - EOF
    - curl ... -d @/tmp/reply.json

Test Coverage Status (All Tests Passing):

- `POST /messages`: Successful creation (201) with `isDeleted: false`.
- `POST /messages (reply)`: Successful threaded reply creation (201) with `parentId` correctly set.
- `GET /messages`: Successful retrieval (200), ensuring paginated results, sorted newest first.
- `PATCH /messages/:id`: Successful message edit (200), setting `isEdited: true`.
- `PATCH /messages/:id (others)`: Correctly blocked with a 403 `E_FORBIDDEN` error.
- `DELETE /messages/:id`: Successful soft deletion (200).
- `GET /messages (after delete)`: Correct retrieval (200), displaying the "This message was deleted" placeholder.
- `DELETE /messages/:id (others)`: Correctly blocked with a 403 `E_FORBIDDEN` error.
- `POST /messages/:id/read`: Successful update (200), confirming `last_read_at` is updated.

Phase 5: Real-time Module (Server-Sent Events)

System Implementation:

- **RealtimeService:** Developed to centralize all Server-Sent Events (SSE) broadcast logic.
- **RealtimeController:** Created to furnish SSE connection information and a server-generated User ID (UID).
- **Dependency Injection:** The `RealtimeService` was injected into the `MessageService` and `ConversationService`.
- **Framework Integration:**
    - The `@adonisjs/transmit` provider was registered within `adonisrc.ts`.
    - Transmit routes were registered using `transmit.registerRoutes()` in `start/routes.ts`.
    - `config/transmit.ts` was added to manage v3 configuration.
- **Inversion of Control (IoC):** The `RealtimeService` IoC binding was established in `AppProvider`.

System Endpoints:

Method	Path	Description
`GET`	`/__transmit/events?uid={uid}`	Initiates a persistent SSE stream connection.
`POST`	`/__transmit/subscribe`	Subscribes the client to a specified channel.
`GET`	`/api/v1/realtime/info`	Retrieves the User ID (UID) and channel subscription instructions.

SSE Channels:

- `conversations/{conversationId}`: Dedicated to all events pertinent to a specific conversation.
- `users/{userId}`: Designated for personal notifications directed to a particular user.

Broadcast Events:

- `message:new`: Broadcast upon the creation of a new message by any participant.
- `message:edited`: Broadcast when the message sender modifies their message content.
- `message:deleted`: Broadcast when the message sender performs a soft deletion of their message.
- `conversation:new`: Broadcast to all involved participants upon the creation of a new conversation.
- `participant:added`: Broadcast when an administrator adds a user to a group

conversation.
- `participant:removed`: Broadcast when a user is removed from a group conversation.

File System Changes (Creation & Update):

File Status	File Path	Notes
Created	`app/services/realtime_service.ts`	
Created	`app/controllers/realtime_controller.ts`	
Created	`config/transmit.ts`	
Updated	`app/services/message_service.ts`	Includes `RealtimeService` injection and broadcast logic.
Updated	`app/services/conversation_service.ts`	Includes `RealtimeService` injection and broadcast logic.
Updated	`providers/app_provider.ts`	Added `RealtimeService` IoC binding.
Updated	`start/routes.ts`	Added `transmit.registerRoutes()` and the `/realtime/info` route.
Updated	`adonisrc.ts`	Registered the `transmit_provider`.

Identified Issues and Resolutions:

Issue Description	Root Cause	Resolution
ERROR 1: `/__transmit/events` resulted in a `404 Not Found`.	Failure to invoke `transmit.registerRoutes()` in `routes.ts`.	**FIX:** `transmit.registerRoutes()` was added to the top of `start/routes.ts`.

ERROR 2: Error message: "Missing required field uid".	Transmit v3 mandates a client-generated UID passed as the query parameter `?uid=CLIENT_UUID`.	**FIX:** The `RealtimeController` was updated to generate and return the UID, which the client subsequently utilizes with `GET /__transmit/events?uid={uid}`.
ERROR 3: Error message: "zsh: bad math expression".	The variable name `UID` is reserved in the Z Shell (zsh) and stores the numeric user ID, leading to the shell interpreting UUID hyphens as arithmetic operators.	**FIX:** All `UID` variable instances within test commands were renamed to `TRANSMIT_UID`.
ERROR 4: `/api/v1/realtime/info` resulted in a `404 Not Found`.	The real-time route group was incorrectly nested within the `/conversations` group, resulting in the actual path being `/api/v1/conversations/realtime/info`.	**FIX:** The real-time group was repositioned outside the conversations group in `routes.ts`.
ERROR 5: The SSE stream immediately terminated with a `":ok"` response.	The `curl` command inherently closes the connection upon receipt of the initial response.	**FIX:** The background process operator (`&`) was implemented in the shell script to maintain the SSE stream connection while subsequent subscription and message sending operations were executed.

Testing Outcomes:

- **`GET /api/v1/realtime/info`:** Successful (HTTP 200) retrieval of UID and channel information.
- **`GET /__transmit/events?uid={uid}`:** Successful opening of the SSE stream (indicated by `: ping`).
- **`POST /__transmit/subscribe`:** Successful channel subscription (HTTP 200).
- **`message:new` event:** Successfully received in real-time via SSE.
- **Phase 1-4 Endpoints:** All prior-phase endpoints remain fully functional.

PHASE 6 — User Presence Module

Implementation Details:

- **PresenceService:** Implemented a Redis-backed presence management system utilizing a heartbeat pattern for status updates.
- **PresenceController:** Developed four API endpoints to manage user online status, offline status, typing indicator, and presence query.
- **`presence_validator.ts`:** Created VineJS validators to ensure the integrity of typing and presence query requests.
- **Updated RealtimeService:** Introduced three new broadcast methods to handle presence-related Server-Sent Events (SSE).
- **Updated AppProvider:** Configured the IoC container to bind the `PresenceService`.
- **Updated `start/routes.ts`:** Defined four presence routes under the `/api/v1/presence` namespace.

API Endpoints:

Method	Path	Description
`POST`	`/api/v1/presence/online`	Registers the user as online (via heartbeat mechanism).
`POST`	`/api/v1/presence/offline`	Explicitly marks the user as offline.
`POST`	`/api/v1/presence/typing`	Sets or clears the user's typing indicator.
`GET`	`/api/v1/presence/:conversationId`	Retrieves the presence status for all participants in a specified conversation.

Redis Key Strategy:

Key Pattern	Value	Time-to-Live (TTL)	Purpose
`presence:online:{userId}`	`"1"`	35 seconds	Indicates active online status.

`presence:typing:{convId}:{userId}`	`"1"`	5 seconds	Indicates user is currently typing in a conversation.
`presence:conversations:{userId}`	Set of conversa…	35 seconds	Tracks active conversations for the online user.

Server-Sent Events (SSE) Broadcasts:

- `presence:online`: Notifies conversation participants when a user transitions to online status.
- `presence:offline`: Notifies conversation participants when a user transitions to offline status.
- `presence:typing`: Notifies conversation participants when a user starts or stops typing.

Core Business Logic:

- The client is required to send a heartbeat request every 20 seconds to maintain online status.
- The **Online TTL is set to 35 seconds** to provide a buffer for network latency.
- The **Typing TTL is set to 5 seconds** to ensure the typing indicator auto-clears promptly upon client disconnection or crash.
- The `presence:online` event is broadcast exclusively upon the *initial* successful heartbeat, preventing excessive broadcasts every 20 seconds.
- Presence query access is restricted solely to confirmed participants of the target conversation.
- Offline status broadcasts are disseminated to all conversations the user was previously participating in.
- All presence routes are secured via authentication middleware.

Files Created/Modified:

- **Created:**
  - `app/services/presence_service.ts`
  - `app/controllers/presence_controller.ts`
  - `app/validators/presence_validator.ts`
- **Updated:**
  - `app/services/realtime_service.ts` (Added 3 presence broadcast methods)
  - `providers/app_provider.ts` (Added `PresenceService` IoC binding)
  - `start/routes.ts` (Added 4 presence routes)

Test Coverage Summary:

All implemented unit and functional tests are passing successfully:

- `POST /presence/online`: Returns 200 OK (Verified `conversationCount: 2`, `expiresIn: 35s`).
- `POST /presence/online` (User: Bob): Returns 200 OK (Verified `conversationCount: 2`, `expiresIn: 35s`).
- `GET /presence/:convId`: Returns 200 OK (Verified both participants are online).
- `POST /presence/typing` (Start): Returns 200 OK (Verified `isTyping: true`).
- `GET /presence/:convId`: Returns 200 OK (Verified Ankit's `isTyping: true` status).
- `POST /presence/typing` (Stop): Returns 200 OK (Verified `isTyping: false`).
- `GET /presence/:convId`: Returns 200 OK (Verified Ankit's `isTyping: false` status).
- `POST /presence/offline`: Returns 200 OK (Verified successful user ID confirmation).
- `GET /presence/:convId`: Returns 200 OK (Verified Ankit offline, Bob online).
- `GET /presence/:convId` (No token): Returns 401 `E_UNAUTHORIZED` (Verified authentication enforcement).

PHASE 7 — Search Module Implementation

Features Developed:

- **SearchService:** Implemented PostgreSQL full-text search capabilities for both messages and conversations.
- **SearchController:** Created three dedicated API endpoints for searching messages, conversations, and performing a global search.
- **search_validator.ts:** Developed VineJS validators to ensure robust input validation for all three search endpoints.
- **AppProvider Update:** Incorporated the `SearchService` through an IoC (Inversion of Control) binding.
- **start/routes.ts Update:** Configured three search routes under the `/api/v1/search` base path.

API Endpoints:

Method	Path	Description
GET	/api/v1/search/messages?q=...	Searches for messages.
GET	/api/v1/search/conversations?q=...	Searches for conversations.
GET	/api/v1/search?q=...	Performs a global search across both messages and conversations.

Search Parameters:

Target	Parameter	Description	Constraints
Messages	q	Search query.	**Required, 1…**
	conversat…	Filters resul…	Optional
	type	Filters by m…	Optional
	dateFrom	Filters mes…	Optional
	dateTo	Filters mes…	Optional
	page	Specifies th…	Optional (D…
	limit	Sets the nu…	Optional (D…
Conversations	q	Search query.	**Required, 1…**

	`type`	Filters by c…	Optional
	`page`	Specifies th…	Optional (D…
	`limit`	Sets the nu…	Optional (D…

Technical Implementation Details:

- **Message Search:**
  - Utilizes PostgreSQL's `tsvector` and `plainto_tsquery` for efficient full-text searching.
  - Employs `ts_rank` for ordering results based on relevance (best matches appear first).
  - Generates search snippets with `<mark>` tags using `ts_headline`.
  - Restricts results to conversations the user is a participant of via a `JOIN` with `conversation_participants`.
  - Excludes soft-deleted messages (`WHERE deleted_at IS NULL`).
- **Conversation Search:**
  - Searches group conversation names using the `ILIKE` operator.
  - Searches for direct conversations by participant names using an `OR` subquery.
  - Employs `DISTINCT` to eliminate duplicate results from the `JOIN` operation.
  - Participants for each result are loaded via a subsequent query.
- **Global Search:**
  - Executes `searchMessages` and `searchConversations` concurrently.
  - Leverages `Promise.all` to ensure optimal performance.
  - Returns a combined result set with unified metadata.

Files Created:

- `app/services/search_service.ts`
- `app/controllers/search_controller.ts`
- `app/validators/search_validator.ts`

Files Updated:

- `providers/app_provider.ts` (Added `SearchService` IoC binding)
- `start/routes.ts` (Added the 3 search routes)

Test Coverage Status (All Tests Passing):

Test Case	Description	Result
`GET /search/messages?q=tomorrow`	Search with a query yielding results.	2 results with `<mark>` snippets.
`GET /search/messages?q=great`	Search for a specific message.	1 result (Bob's message).
`GET /search/messages?q=hello&convId`	Search filtered by a specific conversation ID.	Correctly filtered to the specified conversation.
`GET /search/conversations?q=bob`	Search matching participant names.	2 conversations found (participant match).
`GET /search/conversations?q=Dev`	Search matching a group name.	1 result (group name match).
`GET /search/conversations?q=Dev&type=group`	Search filtered by conversation type.	Type filter executed successfully.
`GET /search?q=meeting`	Global search query.	1 message, 0 conversations returned.
`GET /search/messages?q=xyznotfound`	Search with an intentionally non-existent query.	Empty results, total count: 0.
`GET /search/messages` (no token)	Security test: Missing authorization token.	Returns `401 E_UNAUTHORIZED`.
`GET /search/messages?q=`	Validation test: Empty required query parameter.	Returns `422 E_VALIDATION_ERROR`.

PHASE 8 — File Upload Module

Implementation Summary:

The File Upload Module was developed, encompassing the following core components:

- **`UploadService`:** Implemented for file validation, management of disk storage, and execution of file deletion logic.
- **`UploadController`:** Provides four distinct HTTP endpoints for file upload, retrieval of metadata, and deletion operations.
- **`Upload` Model:** A Lucid ORM model established to interface with the dedicated `uploads` database table.
- **Migration:** A database migration created the `uploads` table, including all necessary columns.
- **`AppProvider` Update:** The `UploadService` was formally bound within the Inversion of Control (IoC) container.
- **`start/routes.ts` Update:** Four specific routes related to the upload functionality were registered under the `/api/v1/uploads` prefix.

Exposed Endpoints:

HTTP Method	URI	Functionality
`POST`	`/api/v1/uploads/im…`	Facilitates the upload of image files.
`POST`	`/api/v1/uploads/fi…`	Facilitates the upload of general file types.
`GET`	`/api/v1/uploads/:f…`	Retrieves the metadata associated with a specified file ID.
`DELETE`	`/api/v1/uploads/:f…`	Executes the deletion of a specified file (restricted to the file owner).

File Constraints and Validation:

Category	Allowed Types	Maximum Size	Validation Methodology
Images	`jpg`, `jpeg`, `png`, `gif`, `webp`	5MB	Validation enforced via MIME type and file extension check.
Files	`pdf`, `doc`, `docx`, `txt`, `zip`	20MB	Validation enforced via MIME type and file extension check.

Storage and Database Strategy:

- **Local Disk Storage:**
  - Images are persisted to: `storage/uploads/images/{uuid}.ext`
  - General files are persisted to: `storage/uploads/files/{uuid}.ext`
  - Files are served publicly from: `/uploads/{category}/{uuid}.ext`
- **Database Record (`uploads` table) Fields:** The database maintains records for `id`, `userId`, `originalName`, `storedName`, `mimeType`, `category`, `disk`, `path`, `url`, and `size`.

Security Measures Implemented:

- **Authorization:** Authentication is mandatory for access to all endpoints.
- **Access Control:** File deletion is restricted solely to the user who performed the original upload (a 403 Forbidden response is returned otherwise).
- **Stored Naming Convention:** Files are stored using UUID-based names to prevent path traversal vulnerabilities.
- **MIME Type Validation:** Validation includes strict checking of the file's MIME type, supplementing the extension check.
- **Extension Whitelisting:** Only explicitly permitted file extensions are accepted.
- **Size Limitations:** File size limits are strictly enforced.

Files Created and Updated:

Category	File Path	Purpose
Created	`app/services/upload_service.ts`	Upload logic service.
Created	`app/controllers/upload_controller.ts`	Endpoint handler.
Created	`app/models/upload.ts`	ORM model.

Created	`database/migrations/..._create_uploads_table.ts`	Database schema definition.
Created	`storage/uploads/images/.gitkeep`	Placeholder for image storage.
Created	`storage/uploads/files/.gitkeep`	Placeholder for file storage.
Updated	`providers/app_provider.ts`	Added IoC binding for `UploadService`.
Updated	`start/routes.ts`	Registered upload-related routes.

Testing Outcomes (All Tests Passed):

- `POST /uploads/file`: **201 Created** (Successful upload of a 30B TXT file, URL generated).
- `POST /uploads/image`: **201 Created** (Successful PNG upload, classified as `image`).
- `GET /uploads/:fileId`: **200 OK** (Metadata retrieval, all fields validated).
- `POST /uploads/image` with a TXT file: **422 Unprocessable Entity** (`E_INVALID_FILE_TYPE` returned, confirming validation).
- `POST /uploads/file` without authentication: **401 Unauthorized** (`E_UNAUTHORIZED` returned, confirming access control).
- `DELETE /uploads/:fileId`: **200 OK** (File successfully deleted from both disk and database).
- `GET /uploads/:fileId` (for a deleted file): **404 Not Found** (`E_FILE_NOT_FOUND` returned).
- `POST message` with file URL: **201 Created** (Message successfully created with `file` type, utilizing the file URL as content).

Phase 9 — Comprehensive Technical Documentation

I. Implemented Features

A. Architecture Overview: Notifications Module

Layer	Component	Functionality
HTTP Layer	`NotificationController`	Provides 5 RESTful endpoints.
Service Layer	`NotificationService`	Contains business logic and manages Server-Sent Events (SSE) broadcast.
Repository Layer	`NotificationRepository`	Handles Create, Read, Update, and Delete (CRUD) operations for PostgreSQL.
Database Layer	`notifications table`	The persistent data store in PostgreSQL.

B. Event Triggers (Automatically Fired by Other Services)

Source Service	Source Method	Triggered Notification Method
`MessageService`	`.send()`	`notifyNewMessage()`
`ConversationService`	`.create()`	`notifyNewConversation()`
`ConversationService`	`.addParticipant()`	`notifyParticipantAdded()`
`ConversationService`	`.removeParticipant()`	`notifyParticipantRemoved()`

II. Codebase Changes

A. Files Created

Path	File	Purpose
`app/models/`	`notification.ts`	Lucid ORM Model for Notifications.

app/services/	notification_service.ts	Implementation of core business logic.
app/controllers/	notification_controller.ts	HTTP request handling layer.
app/repositories/	notification_repository.ts	Concrete implementation of database queries.
app/repositories/interfaces/	i_notification_repository.ts	Interface for Dependency Injection (DI).
database/migrations/	xxxx_create_notifications_table.ts	Database schema migration file.

B. Files Modified

Path	File	Reason for Modification
app/models/	conversation_participant.ts	Added explicit columnName mappings.
app/services/	message_service.ts	Corrected method signature and integrated notification trigger.
app/services/	conversation_service.ts	Integrated notification triggers for life-cycle events.
app/services/	realtime_service.ts	Implemented broadcastNotification() functionality.
app/services/inter…	i_message_service.ts	Updated .send() method signature.
app/repositories/	conversation_repository.ts	Converted to use 100% raw database queries.

`providers/`	`app_provider.ts`	Registered and bound `NotificationService`.
`start/`	`routes.ts`	Defined 5 new notification routes.

III. Database Schema

The `notifications` table schema is defined as follows:

```
CREATE TABLE notifications (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  actor_id        UUID REFERENCES users(id) ON DELETE SET NULL,
  type            ENUM(
                  'message:new',
                  'conversation:new',
                  'participant:added',
                  'participant:removed'
                ) NOT NULL,
  conversation_id  UUID,
  message_id       UUID,
  title           VARCHAR NOT NULL,
  body            VARCHAR NOT NULL,
  is_read         BOOLEAN NOT NULL DEFAULT false,
  read_at         TIMESTAMP,
  created_at      TIMESTAMP NOT NULL,
  updated_at      TIMESTAMP NOT NULL,
```

```
  INDEX (user_id, is_read),
  INDEX (user_id, created_at)
);
```

IV. API Endpoints

1. **List Notifications**
    - **Method:** `GET /api/v1/notifications`
    - **Authorization:** Bearer Token required.
    - **Query Parameters:** `page` (number, default: 1), `limit` (number, default: 20, max: 50), `isRead` (boolean, optional filter).
    - **Successful Response (200):** Returns a paginated list of notifications and metadata, including `unreadCount`.
2. **Get Unread Count**
    - **Method:** `GET /api/v1/notifications/unread-count`
    - **Authorization:** Bearer Token required.
    - **Successful Response (200):** Returns an object containing the total `unreadCount`.
3. **Mark Single as Read**
    - **Method:** `PATCH /api/v1/notifications/:id/read`
    - **Authorization:** Bearer Token required.
    - **Successful Response (200):** Confirms the notification is marked as read and provides updated data.
    - **Error Response (404):** `E_NOTIFICATION_NOT_FOUND`.
4. **Mark All as Read**
    - **Method:** `PATCH /api/v1/notifications/read-all`
    - **Authorization:** Bearer Token required.
    - **Successful Response (200):** Confirms the operation and provides the `count` of notifications marked as read.
5. **Delete Notification**
    - **Method:** `DELETE /api/v1/notifications/:id`
    - **Authorization:** Bearer Token required.
    - **Successful Response (200):** Confirms the deletion of the notification.
    - **Error Response (404):** `E_NOTIFICATION_NOT_FOUND`.

V. Notification Trigger Logic

Trigger No.	Event Description	Triggering Component	Target Recipients	Notification Type	Title/Body Format	SSE Broadcast Channel

1	New Message	`MessageService.send()`	All conversation participants EXCEPT sender.	`message:new`	Title: New message from {senderName} / in {conversationName}. Body: First 80 chars of message.	`user…`
2	New Conversation	`ConversationService.create()`	All participants EXCEPT creator.	`conversation:new`	Title: {creatorName} added you to a conversation. Body: Added to {name} / New direct message from {name}.	`user…`
3	Participant Added	`ConversationService.addParticipant()`	The added user only.	`participant:added`	Title: Added to {conversationName}. Body: {actorName} added you to {conversationName}.	`user…`
4	Participant Removed	`ConversationService.removeParticipant()`	The removed user only.	`participant:removed`	Title: Removed from {conversationName}. Body: {actorName} removed	`user…`

					you from {conversationName}.	

VI. Security and Reliability

- **Authorization:** Authentication is strictly enforced on all notification endpoints.
- **Access Control:** Users are restricted to viewing, marking, or deleting only their own notifications.
- **Data Integrity (User Deletion):** Notifications are automatically deleted upon user deletion (via `ON DELETE CASCADE`).
- **Data Integrity (Actor Deletion):** The `actor_id` reference is set to NULL upon actor deletion.
- **System Reliability:** The notification trigger mechanism operates on a fire-and-forget principle, ensuring that message transmission is non-blocking and is not affected by notification errors.

VII. Resolved Defects

- **BUG 1: MessageService.send() Signature Mismatch**
  - **Issue:** The controller and service layer had incompatible method signatures for `MessageService.send()`.
  - **Resolution:** The `MessageService.send()` method was refactored to accept an object as the first parameter for data encapsulation.
- **BUG 2: ConversationRepository ORM Misconfiguration**
  - **Issue:** `ConversationRepository` utilized the Lucid ORM model for a join table without explicit column name mappings, leading to binding errors.
  - **Resolution:** All model-based queries were replaced with raw `db.from()` and `whereRaw()` database queries.
- **BUG 3: NotificationService Dynamic Import Issues**
  - **Issue:** Dynamic imports within an asynchronous method in `NotificationService` caused timing-related errors.
  - **Resolution:** The dynamic import was replaced with a direct raw database query for participant lookup.
- **BUG 4: IMessageService Interface Drift**
  - **Issue:** The `IMessageService` interface contained outdated method signatures.
  - **Resolution:** All method signatures within the interface were updated to accurately reflect the current implementation.
- **BUG 5: AppProvider Dependency Injection Error**
  - **Issue:** `AppProvider` incorrectly injected `conversationRepository` into `MessageService` (which was no longer required).
  - **Resolution:** The dependency binding was corrected to pass only

`messageRepository`, `realtimeService`, and `notificationService`.

VIII. Testing Results

All test cases for the notification functionality passed successfully.

Test Case	Description	Expected Outcome	Status
1	`GET /notific…`	`unreadCount:…`	✅
2	Bob sends a m…	Ankit receives …	✅
3	Bob sends a se…	Ankit receives …	✅
4	`GET /notific…`	`unreadCount:…`	✅
5	`GET /notific…`	Returns 3 resul…	✅
6	`GET /notific…`	Correctly filters …	✅
7	`PATCH /notif…`	Notification mar…	✅
8	`GET /notific…`	Returns 1 read …	✅
9	`PATCH /notif…`	2 notifications …	✅
10	`GET /notific…`	`unreadCount:…`	✅
11	`DELETE /noti…`	Notification del…	✅
12	Ankit sends a …	Bob's unreadC…	✅
13	`GET /notific…`	Returns 401 E_…	✅

Phase 10 — Comprehensive Technical Documentation

Architecture Overview

The system employs a multi-layered middleware architecture, with a primary focus on rate limiting and security enforcement.

RATE LIMITING + SECURITY LAYER
Applied to Every Request:
1. `ContainerBindingsMiddleware` (Dependency Injection setup)
2. `BodyParserMiddleware` (JSON body parsing)
3. `CorsMiddleware` (Cross-Origin Resource Sharing configuration)
4. `SecurityHeadersMiddleware` (Application of security response headers)
$\downarrow$
Applied to Specific Routes (Selectively):
1. `RateLimitMiddleware` (Checks the Redis counter for request limits)
2. `AuthMiddleware` (Validates the JSON Web Token)
3. `ConversationAccess` (Verifies participant authorization)
$\downarrow$
Rate Limit Flow:
Request Initiation $\rightarrow$ Identity Extraction (IP/UserId) $\rightarrow$ Redis Key Generation $\rightarrow$ Atomic Counter Increment $\rightarrow$ Limit Evaluation $\rightarrow$ Header Setting $\rightarrow$ Access Granted (200) $\text{ or } $ Access Denied (429)

Files Created

Path	Description
`app/middleware/rate_limit_middleware.ts`	Implements rate limit enforcement logic.
`app/middleware/security_headers_middleware.ts`	Implements security header application.

`app/services/rate_limit_service.ts`	Contains the Redis sliding window algorithm logic.
`config/rate_limit.ts`	Centralized configuration for rate limit rules.

Files Modified

Path	Description
`start/kernel.ts`	Inclusion of `BodyParser` and security middleware.
`start/routes.ts`	Application of `RateLimitMiddleware` to designated routes.

Rate Limit Rules Matrix

ROUTE	LIMIT	WINDOW	KEY TYPE
`POST /auth/login`	5	15 min	IP address
`POST /auth/register`	3	60 min	IP address
`POST /auth/guest`	10	60 min	IP address
`POST /auth/refresh`	20	1 min	IP address
`POST /uploads/image`	20	60 min	User ID
`POST /uploads/file`	20	60 min	User ID
`POST /conversations/:id/msgs`	60	1 min	User ID

Key Strategy:

- **Authenticated Routes:** Keyed by `userId` to ensure fair, per-user limits.
- **Authentication Routes:** Keyed by IP address to mitigate automated attacks (bots/scrapers).
- **Fallback:** Keyed by IP address for all unauthenticated requests.

Rate Limiting Implementation Details

The system employs the **Sliding Window Algorithm**.

Example (window = 60 seconds, limit = 5):

Time	Request	Counter	Status	Remaining
t=0s	Request 1	1	ALLOW $\…	4
t=10s	Request 2	2	ALLOW $\…	3
t=20s	Request 3	3	ALLOW $\…	2
t=30s	Request 4	4	ALLOW $\…	1
t=40s	Request 5	5	ALLOW $\…	0
t=50s	Request 6	6	BLOCK $\t…	429 returned
t=60s	Window resets	0	ALLOW $\…	-

Redis Key Structure

The pattern for Redis keys is: `{appPrefix}:{rulePrefix}:{keyType}:{identifier}`

Examples:

- `convocore:rl:auth:login:ip:172.18.0.1`
- `convocore:rl:message:user:e542a490-001d-4578-8e9d-9299868f83da`
- `appPrefix`: Configured in `config/redis.ts`'s `keyPrefix`.
- `rulePrefix`: Configured in `config/rate_limit.ts`.

- `keyType`: Either "ip" or "user".
- `identifier`: The specific IP address or `userId`.

Atomic Pipeline Operation

```
// Uses Redis pipeline for atomic INCR + TTL to prevent race conditions

const pipeline = redis.pipeline()

pipeline.incr(redisKey)    // atomic increment

pipeline.ttl(redisKey)     // get time to live

const results = await pipeline.exec()


// Sets expiry ONLY for the first request (when counter equals 1 and ttl < 0)

if (ttl < 0) {

  await redis.expire(redisKey, windowSec)

}
```

Security Headers Implementation

Headers Applied to Every Response:

HEADER	VALUE	PROTECTION AGAINST
`X-Content-Type-Options`	`nosniff`	MIME sniffing attacks
`X-Frame-Options`	`DENY`	Clickjacking
`X-XSS-Protection`	`1; mode=block`	Cross-Site Scripting (legacy)
`Referrer-Policy`	`strict-origin-when-cross-origin`	Data leakage
`Permissions-Policy`	`camera=(),`	Device/feature abuse

	`microphone=(), geolocation=(), payment=()`	
`Content-Security-Policy`	`default-src 'self', script-src 'self', frame-ancestors none`	XSS and injection attacks
`X-Powered-By`	REMOVED	Server fingerprinting

Additional Headers for API Routes (`/api/*`):

HEADER	VALUE	PURPOSE
`Cache-Control`	`no-store, no-cache, must-revalidate`	Prevents data caching
`Pragma`	`no-cache`	HTTP/1.0 cache control
`Expires`	`0`	Mitigates proxy caching

Production-Only Header:

HEADER	VALUE	PURPOSE
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains; preload`	Enforces HTTPS usage

Rate Limit Response Headers

On a Successful (200) Response for a Rate-Limited Route:

HTTP/1.1 200 OK

X-RateLimit-Limit: 60

X-RateLimit-Remaining: 59

X-RateLimit-Reset: 1778142315

- `X-RateLimit-Limit`: The total number of requests permitted in the window.
- `X-RateLimit-Remaining`: The number of requests remaining before blocking.
- `X-RateLimit-Reset`: The Unix timestamp when the window resets.

On a Blocked (429) Response (Too Many Requests):

HTTP/1.1 429 Too Many Requests

X-RateLimit-Limit: 5

X-RateLimit-Remaining: 0

X-RateLimit-Reset: 1778142315

Retry-After: 899

Content-Type: application/json

{

"message": "Too many requests. Please try again later.",

"code": "E_RATE_LIMIT_EXCEEDED",

"retryAfter": 899,

"resetAt": "2026-05-07T08:25:14.045Z"

}

Middleware Execution Sequence

The following sequence dictates the order of middleware execution for an incoming request:

Order	Middleware	Function	Notes
1	`ContainerBindingsMiddleware`	Initializes the IoC container setup.	

2	`BodyParserMiddleware`	Parses the request body (JSON/form).	**CRITICAL:** Must execute early as subsequent middleware (e.g., Rate Limiter) may rely on parsed body content.
3	`CorsMiddleware`	Handles Cross-Origin Resource Sharing, including `OPTIONS` preflight requests.	
4	`SecurityHeadersMiddleware`	Injects security headers into the response.	
5	`RateLimitMiddleware`	Checks the Redis counter for rate limit enforcement.	Route-specific application.
6	`AuthMiddleware`	Validates the JWT token.	Route-specific application.
7	`ConversationAccessMiddleware`	Confirms user participation access.	Route-specific application.
8	Controller	Executes the final route logic.	

Bugs Addressed in This Phase

Issue	Symptom	Cause	Resolution
1. Body parser not registered	Validation error: "email field is required" despite the field being present in the request.	`BodyParserMiddleware` was omitted from `kernel.ts`, causing the Rate Limit middleware to execute before the request body was parsed.	Added `@adonisjs/core/bodyparser_middleware` to `server.use()` *before* the Rate Limit middleware.

2. Redis key prefix mismatch	Redis `clear` command reported 0 keys deleted, and rate limits persisted.	AdonisJS Redis client automatically prepends `"convocore:"` to all keys. Our clear pattern (`"rl:*"`) did not match the actual key (`"convocore:rl:auth:login:ip:172.18.0.1"`).	Updated the clear pattern to `"convocore:rl:*"`. `RateLimitService` key prefix remains `"rl:auth:login"` as the Redis client handles the application prefix.
3. `X-Powered-By` set to empty string	The header was still present in responses as `x-powered-by:`.	Using `res.header('X-Powered-By', '')` sets an empty value but does not remove the header.	Changed implementation to `res.response.removeHeader('X-Powered-By')`, utilizing the Node.js native method for header removal.
4. Rate limit persistence in tests	Login attempts were blocked even after the test script attempted to clear Redis data.	The test script was executed from the temporary directory (`/tmp`), and the `docker compose exec` command required the context of the main `ConvoCore` directory to function correctly.	Added `cd ~/Desktop/ConvoCore` at the beginning of the test script to ensure correct execution context.

Phase 12:
ConvoCore — Integration Test Suite Analysis

ConvoCore — Integration Test Suite Analysis

I. Overview

This document presents a comprehensive analysis of the complete integration test suite for ConvoCore, leveraging the provided search results and covering all four designated domains.

II. Project Structure

```
tests/
├── integration/
│   ├── auth/                # Registration, login, guest sessions
│   ├── conversations/       # Creation, access control, CRUD operations
│   ├── messages/            # Sending, replies, ownership validation
│   ├── notifications/       # Realtime WebSocket communication
│   └── security/            # Middleware and authorization enforcement
├── helpers/
│   ├── factory.ts           # Test data creation utilities
│   └── auth.ts              # Authentication helper functions
└── setup.ts                 # Global database transaction setup
```

III. Core Infrastructure

A. Global Setup — Transaction Isolation

The setup module ensures complete test isolation by executing each test within a dedicated database transaction that is subsequently rolled back, preventing any data leakage.

```
// tests/setup.ts

group.each.setup(async () => {

  await Database.beginGlobalTransaction()
```

```
  return () => Database.rollbackGlobalTransaction()
})
```

B. Factory Helper

The `Factory` object facilitates the standardized creation of test entities.

```
// tests/helpers/factory.ts
export const Factory = {
  createUser(overrides?)              // Creates user with a hashed password
  createConversation(creatorId, participantIds[])  // Automatically attaches participants
  createMessage(conversationId, userId, content, replyToId?)
  createGuest()                       // Creates a guest user with null email/password
}
```

C. Auth Helper

Authentication utilities centralize token management and header construction.

```
// tests/helpers/auth.ts
loginAs(email, password)              // Returns { accessToken, refreshToken }
authHeader(token)                     // Returns { Authorization: 'Bearer <token>' }
```

IV. Authentication Tests

A. Registration — `register.spec.ts`

Test Case	Expected Status	Critical Assertion
Valid registration	201 (Created)	Returns user object and tokens; password field is absent.

Duplicate email	409 (Conflict)	Error message explicitly mentions "email".
Missing required fields	422 (Unprocessable En…	`body.errors` is a non-empty array.
Weak password policy violation	422 (Unprocessable En…	Validation is correctly rejected.

Security Assertion: Password field must be absent from the response payload.

assert.notExists(body.data.user.password)

assert.exists(body.data.accessToken)

assert.exists(body.data.refreshToken)

B. Login — `login.spec.ts`

Test Case	Expected Status	Key Assertion
Valid credentials	200 (OK)	Both access and refres…
Incorrect password	401 (Unauthorized)	Response message ind…
Non-existent user	401 (Unauthorized)	—
Token refresh validity	200 (OK)	New token value differs…
Invalid refresh token	401 (Unauthorized)	—

Token Rotation Verification:

assert.notEqual(body.data.accessToken, loginBody.data.accessToken)

C. Guest Sessions — `guest.spec.ts`

Test Case	Expected Status	Key Assertion
Guest token creation	201 (Created)	`guestToken` exists; `isGuest` flag is true.
Session uniqueness	201 (Created) (x2)	Two consecutive tokens are distinct.

V. Conversation Tests

A. Creation — `create.spec.ts`

Assertion: Participant count includes the creator and all invited users.

assert.lengthOf(body.data.participants, 3) // owner + p1 + p2

Test Case	Expected Status
Create with participants	201 (Created)
Missing authentication header	401 (Unauthorized)
Missing required 'name' field	422 (Unprocessable Entity)

B. Access Control — `access.spec.ts`

Critical Security Requirement: Access or modification of a conversation is restricted exclusively to its participants.

Test Case	Expected Status
Participant retrieves own conversation	200 (OK)
Non-participant attempts GET access	403 (Forbidden)
Non-participant attempts PATCH modification	403 (Forbidden)
Request for a non-existent conversation	404 (Not Found)

C. CRUD Operations — `crud.spec.ts`

Scoped Listing: Users are permitted to view only conversations in which they are a participant.

assert.lengthOf(body.data, 2) // userA sees exactly 2

Test Case	Expected Status	Key Assertion
List own conversations	200 (OK)	Results are correctly filtered.
Update conversation name	200 (OK)	Resource reflects the updated name.
Delete conversation	200 (OK) then 404 (Not…	Subsequent retrieval fails.

VI. Message Tests

A. Sending — `send.spec.ts`

Critical Security Requirement: Only participants are authorized to read or send messages within a conversation.

Test Case	Expected Status	Key Assertion
Participant sends message	201 (Created)	`userId matches the…`
Non-participant sends message	403 (Forbidden)	—
Empty message content	422 (Unprocessable Entity)	—
Paginated listing	200 (OK)	`data.length is 10;…`

Pagination Metadata Validation:

assert.lengthOf(body.data, 10)

assert.exists(body.meta.total)

assert.equal(body.meta.total, 15)

B. Ownership — `ownership.spec.ts`

Critical Security Requirement: Users are permitted to modify only their own messages.

Test Case	Expected Status
Owner updates own message	200 (OK)
Other participant updates a foreign message	403 (Forbidden)
Owner deletes own message	200 (OK)
Other participant deletes a foreign message	403 (Forbidden)

C. Replies — `replies.spec.ts`

Assertion: A reply response must include the full referenced message object.

assert.equal(body.data.replyToId, original.id)

assert.exists(body.data.replyTo)

assert.equal(body.data.replyTo.content, 'Original message')

Test Case	Expected Status	Scenario
Reply to a valid message	201 (Created)	The response includes the `replyTo` object.
Reply to a non-existent ID	422 (Unprocessable En…	Invalid reference.
Cross-conversation reply	422 (Unprocessable En…	Referenced message belongs to a different conversation.

VII. Realtime / Notification Tests

These tests utilize native WebSocket connections to verify the reliable broadcast of events to conversation participants.

// tests/integration/notifications/realtime.spec.ts

import WebSocket from 'ws'

```
import { WS_URL } from '../../setup'

// Pattern: connect participant WS → send HTTP message → assert WS event received
test('broadcasts message to all conversation participants', async ({ assert }) => {
  const ws = new WebSocket(`${WS_URL}/conversations/${id}?token=${accessToken}`)

  ws.on('message', (data) => {
    const event = JSON.parse(data.toString())
    assert.equal(event.type, 'new_message')
    assert.equal(event.data.content, 'Hello World!')
  })

  // Message is sent via HTTP and delivery is expected via WebSocket
  await supertest(BASE_URL)
    .post(`/api/conversations/${id}/messages`)
    .set(authHeader(token))
    .send({ content: 'Hello World!' })
})
```

VIII. Security Tests

A. Middleware — `middleware.spec.ts`

Authentication middleware must be enforced on all protected routes.

```
// Every protected route must enforce JWT auth
```

```
const protectedRoutes = [
  ['GET',    '/api/conversations'],
  ['POST',   '/api/conversations'],
  ['GET',    '/api/conversations/1/messages'],
  ['POST',   '/api/conversations/1/messages'],
]

// Verification that each route returns 401 when no token is provided
for (const [method, path] of protectedRoutes) {
  const res = await supertest(BASE_URL)[method.toLowerCase()](path)
  assert.equal(res.status, 401)
}
```

B. Authorization — `authorization.spec.ts`

Resource access must be strictly controlled; a valid token does not grant access to resources owned by another user.

```
test('cannot access other user resources with valid but wrong token', async () => {
  const tokenA = await loginAs('userA@test.com', 'Pass@1234')
  const conversationOwnedByB = await Factory.createConversation(userB.id)

  const { status } = await supertest(BASE_URL)
    .get(`/api/conversations/${conversationOwnedByB.id}`)
    .set(authHeader(tokenA.accessToken))

  assert.equal(status, 403)  // Correct status for valid token, insufficient permissions
```

})

IX. HTTP Status Code Reference

Status Code	Meaning	Description
201	Created	Successful resource creation.
200	OK	Successful read, update, or deletion.
401	Unauthorized	Missing or invalid JSON Web Token (JWT).
403	Forbidden	Valid token, but insufficient permissions for the resource.
404	Not Found	Requested resource does not exist.
409	Conflict	Attempt to create a duplicate resource (e.g., existing email).
422	Unprocessable Entity	Request failed due to validation errors.

X. Key Design Principles

- **Test Isolation:** Achieved through comprehensive database transaction rollback per test.
- **Code Efficiency:** Factory pattern is implemented to minimize repetitive test setup logic.
- **Authentication Management:** Centralized via dedicated helper functions for token operations.
- **Error Semantics:** Strict adherence to the 401 (Unauthorized) vs. 403 (Forbidden) distinction.
- **Data Security:** Passwords are never serialized in API responses.
- **Pagination Fidelity:** Responses include both the data array and the `meta.total` count.
- **Reply Integrity:** Validation prevents cross-conversation message references.
- **Realtime Verification:** WebSocket broadcast functionality is confirmed through parallel connection testing.

Phase 12 — Comprehensive Delivery Summary

File	Purpose
`docker/app/Dockerfile`	Multi-stage build configuration
`docker/nginx/nginx.conf`	Primary Nginx configuration file
`docker/nginx/conf.d/convocore.conf`	Application server block and SSL configuration
`docker-compose.prod.yml`	Production environment orchestration definition
`.env.production.example`	Template for production environment variables
`.env.example`	Template for development environment variables
`.github/workflows/ci.yml`	Continuous Integration (CI) pipeline definition
`.github/workflows/deploy.yml`	Continuous Deployment (CD) auto-deployment workflow
`deploy.sh`	Manual deployment execution script

Overall Project Status

Phase	Description	Status
1-10	Core Functionality Implementation	$\checkmark$ Complete
11	Automated Testing Suite	$\checkmark$ Complet…
12	API Documentation Generation	$\checkmark$ Complete
13	Performance Optimization and Caching	$\checkmark$ Complete
14	Production Deployment Setup	$\checkmark$ Complete

Subsequent Steps for Virtual Private Server (VPS) Deployment

The following procedure outlines the deployment process upon acquisition of a VPS:

1. Establish SSH connection to the VPS

ssh user@your-vps-ip

2. Install required dependencies (Docker and Git)

sudo apt update && sudo apt install -y docker.io docker-compose-plugin git

3. Clone the repository to the designated directory

git clone https://github.com/yourusername/convocore.git /opt/convocore

cd /opt/convocore

4. Configure environment variables

cp .env.production.example .env.production

nano .env.production # Populate all required variable values

5. Initiate deployment

./deploy.sh --fresh

Configuration of GitHub Secrets for Automated Deployment

To enable Continuous Deployment via GitHub Actions, the following secrets must be configured within the repository settings (GitHub → Settings → Secrets → Actions → New repository secret):

Secret Name	Description/Value
`DOCKERHUB_USERNAME`	Your Docker Hub user identifier

`DOCKERHUB_TOKEN`	Your Docker Hub personal access token
`PROD_HOST`	The IP address of your VPS
`PROD_USER`	The SSH username for your VPS (e.g., `ubuntu`)
`PROD_SSH_KEY`	Your private SSH key associated with the VPS
`PROD_PORT`	The SSH port (Default: `22`)